

LAB 03

Running

Running: Aerial Gait with Intended Progress

Environment design · PPO · Honest verification

BUILD → TRAIN → VERIFY → EXPLAIN



What you will build — and what the evidence says

- 01 Use case: Teach why fast motion is not enough: a running policy must survive, follow the commanded direction, alternate support, and show true both-feet aerial phases.
- 02 By the end: describe the scene, map observations to actions, explain the reward, trace one PPO update, run a bounded experiment, and defend an evaluation claim.
- 03 The policy specializes to one nominal 0.75 m/s forward command; arbitrary speed, turning, and disturbance recovery are untested.
- 04 Scope: local MuJoCo simulation. Do not deploy these classroom artifacts to a physical robot; hardware transfer requires a separate validated process.

Materials and prerequisites · 1/2

- 01 Apple Silicon Mac with installed RLX/MLX, MuJoCo, ONNX Runtime, rendering dependencies, and Node.
- 02 Tracked base and refinement recipes: running-base.json and running.json.
- 03 Studio trainer/exporter, scenario HTTP driver, independent audit_scenarios.py, and video verifier.
- 04 Saved base checkpoint plus adjacent normalization sidecar; both are required for the refinement stage.

Materials and prerequisites · 2/2

05

Final checkpoint, ONNX actor, evaluation JSON, physical NPZ traces, optimizer JSONL, MP4, contact sheet, and render receipt.

Environment and policy contract · 1/2

- 01 Nominal XML-actuator MuJoCo at 50 Hz; fixed command is 0.75 m/s forward, zero lateral speed, and zero yaw rate.
- 02 Observation is float32 batch x 61 with gyro, projected gravity, 14 joint offsets, lagged velocities, prior raw action, and 13 command values.
- 03 Absolute world position and yaw are absent; gyro yaw rate and commanded yaw rate are observable, so yaw-rate tracking is learnable.
- 04 Action is float32 batch x 14 raw target offsets; the environment stores raw values, clips applied offsets to [-4,4], and adds DEFAULT_POSE.

Environment and policy contract · 2/2

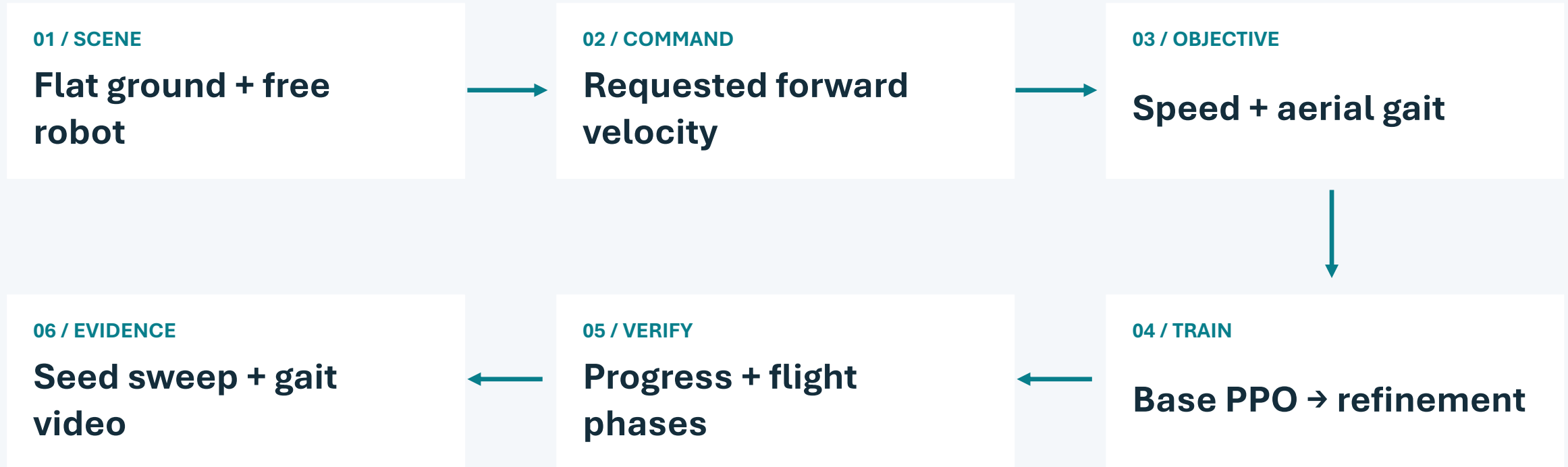
05

Episode horizon is 600 control steps or 12 seconds; domain randomization, observation noise, action delay, and random yaw are disabled.

06

The native speed curriculum is bypassed by pinning every twist command to $[0.75, 0, 0]$; head/body keep-alive commands still vary.

How this case is built: one evidence loop



Failure is data: change one cause, retrain, and repeat the same evaluation gates.

Build the case step by step · 1/2

- 01 Train Running v3 from random actor and critic weights for 6,000,640 transitions using the base reward and fixed 0.75 m/s command.
- 02 Retain the v3 failed intended-progress verdict; it acquired an aerial gait but curved enough to fail signed command-directed displacement.
- 03 Copy both the v3 checkpoint and its .safetensors.json normalizer sidecar into a fresh v4 run directory.
- 04 Resume for 2,097,152 transitions with lower combined angular weight and new observable yaw-rate-only shaping.

Build the case step by step · 2/2

05

Export the deterministic actor with normalization baked in; do not substitute a raw checkpoint or mismatched sidecar.

06

Evaluate 12-second episodes with heading-anchored signed displacement, support events, foot air/contact fractions, and aerial fraction.

07

Audit seeds 501-505 against zero, random, and initial controls, then validate ONNX parity, complete MP4 decoding, and browser playback.

PPO: improve the actor without trusting a huge step

$$\text{ratio} = \pi_{\text{new}}(a \mid o) / \pi_{\text{old}}(a \mid o)$$

Maximize $\text{mean}[\min(\text{ratio} \times A, \text{clip}(\text{ratio}, 1 - \epsilon, 1 + \epsilon) \times A)]$

Actor: chooses joint actions. Critic: estimates future discounted reward.

A = advantage. ϵ = clipping range. Old log probabilities stay fixed for each collected batch.

Follow one rollout through the learner

- 01 Collect observations, sampled actions, old log probabilities, values, rewards and episode-boundary flags using the current stochastic actor.
- 02 Compute TD residuals and generalized advantage estimates (GAE). A true terminal has zero continuation value; a timeout may bootstrap the saved final observation.
- 03 Cut advantage traces at reset boundaries. Never bootstrap a timeout from the first observation of the next episode.
- 04 Reuse this batch for PPO minibatch updates, then collect fresh on-policy data. Export the deterministic actor and its observation transform for evaluation.

Paper exercise: count samples and clip the objective

- 01 Illustrative arithmetic, NOT this run: $16 \text{ environments} \times 128 \text{ steps} = 2,048$ fresh transitions per rollout. Four epochs reuse those transitions; they do not create 8,192 new samples.
- 02 With $\epsilon = 0.2$, ratio = 1.4 and $A = +2$: $\min(2.8, 2.4) = 2.4$. The minimized policy term is -2.4 before averaging and additional loss terms.
- 03 With the same ratio and $A = -2$: $\min(-2.8, -2.4) = -2.8$. Explain why clipping is asymmetric once the sign of the advantage changes.
- 04 Run from repo root: `python3 docs/microduck-training-student-book/learning_lab.py --gae`. This is a deterministic arithmetic lab, not new policy training.

Reward design: what the agent is paid to do · 1/2

- 01 keep_pace 12: baseline-subtracted Gaussian command tracking, so standing at a nonzero command scores zero.
- 02 track_turn 2: combined yaw, roll, and pitch rate tracking; reduced from 8 in the base recipe.
- 03 yaw_tracking 8: focused Gaussian on observable gyro yaw rate versus commanded yaw rate.
- 04 flight 4: rewards both feet airborne only while upright and moving in the commanded direction; air_time 1 rewards per-foot flight windows.

Reward design: what the agent is paid to do · 2/2

05

stay_upright 2, pose 0.1, and head_up 1 shape posture without defining the independent running gate.

06

foot_clearance 0.5, smooth_moves 0.1, plant_the_foot 0.05, no_limit_parking 1, and calm_roll 0.025 are active costs.

Recorded training recipe · 1/2

- 01 Base v3: seed 7, 6,000,640 transitions, 16 envs x 128 steps, 2,930 rollouts, 46,880 optimizer minibatches, learning rate 0.0003.
- 02 Continuation v4: 2,097,152 transitions, 1,024 rollouts, 16,384 optimizer minibatches, learning rate 0.0001.
- 03 Entire selected lineage: 8,097,792 transitions, 3,954 rollout/update cycles, and 63,264 optimizer minibatches.
- 04 Both stages use 4 minibatches x 4 epochs, gamma 0.99, GAE lambda 0.95, clip 0.2, value coefficient 1, entropy 0, and max grad norm 0.5.

Recorded training recipe · 2/2

05

Initial std is 0.6 in the base; continuation loads learned actor, critic, log std, observation stats, and reward-return stats but fresh Adam state.

06

Update log fields are phase, steps, seconds, optimizer_steps, mean_loss, policy_loss, value_loss, entropy, approximate_kl, clip_fraction, explained_variance, env_steps.

Hands-on commands: inspect, smoke, train · 1/3

01

```
From .:  
./scripts/train-running.sh --dry-run  
Inspect the guarded two-stage Running pipeline without training.
```

02

```
From .:  
./scripts/train-running.sh  
Run the independent Full Running base and refinement pipeline with fresh outputs.
```

Hands-on commands: inspect, smoke, train · 2/3

03

```
From .:  
RUN=running-classroom-$(date +%Y%m%d-%H%M%S); ./scripts/train-running.sh --recipe-json  
docs/remaining-scenarios-e2e/recipes/running.json --init-from  
rlx/runs/studio/running/running-e2e-20260907-v3/running.safetensors --run-name "$RUN"  
Refine the retained v3 checkpoint with its adjacent sidecar in a fresh run; keep RUN in  
this terminal for the following audit. Requires the ignored historical base artifacts.
```

04

```
From .:  
rlx/.venv-microduck/bin/python rlx/scripts/audit_scenarios.py --recipe-json docs/remaining-  
scenarios-e2e/recipes/running.json --run "rlx/runs/studio/running/${RUN:?Run the refinement  
command first in this shell}" --output "rlx/artifacts/$RUN-audit" --seeds 501 502 503 504  
505 --render  
Audit a completed Running reproduction over five held-out reset seeds.
```

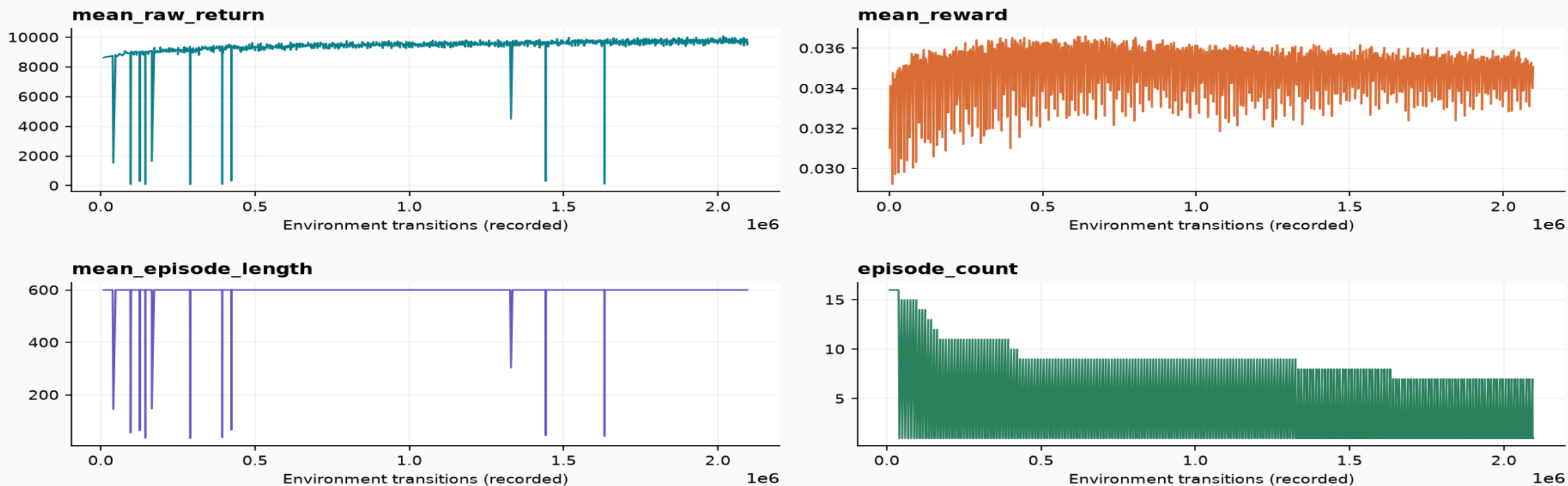
Hands-on commands: inspect, smoke, train · 3/3

05

```
From .:  
python3 -m unittest discover -s scripts/scenarios -p 'test_*.py'  
Run scenario-pipeline unit tests without starting training.
```

Reward: does the logged signal match the task?

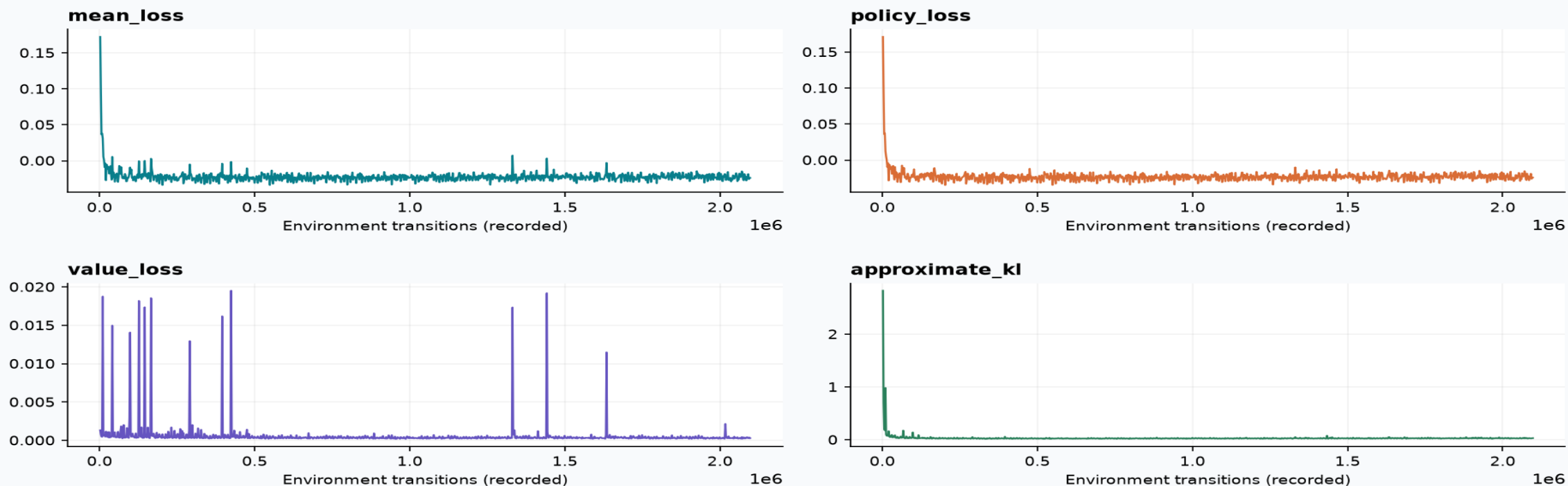
Recorded reward signals — different definitions, not interchangeable



Training reward is an optimization signal, not the acceptance test. See [raw/<case>/](#) for complete recorded fields.

Loss: inspect actor, critic and policy drift

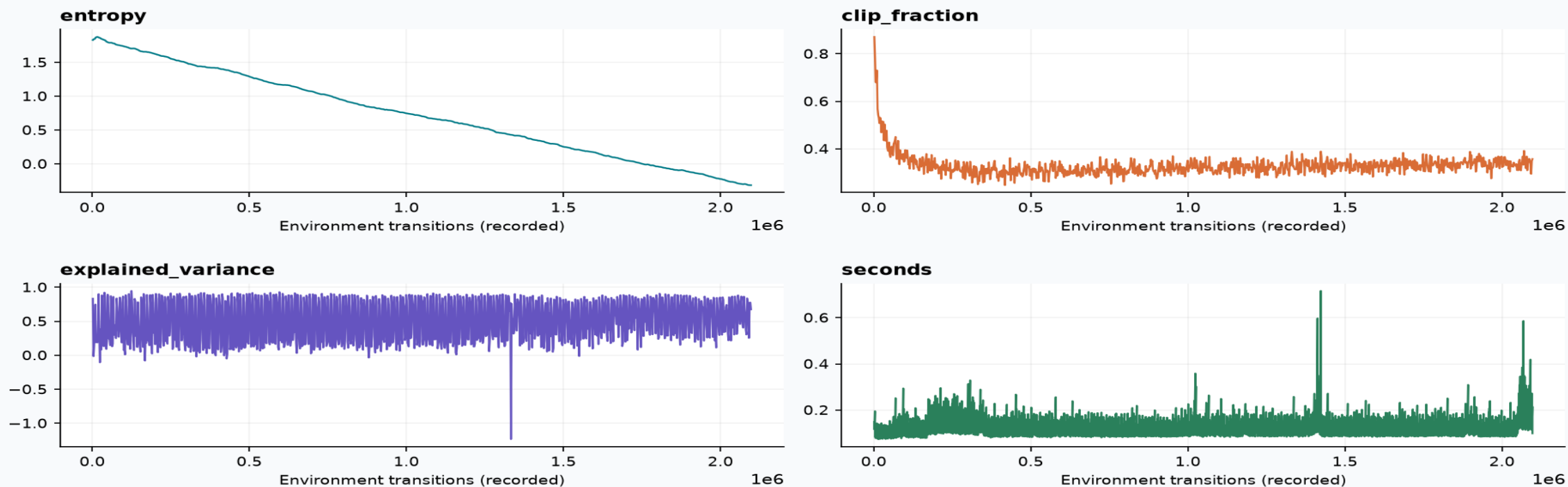
Recorded PPO optimization — raw values, no smoothing



A lower loss does not prove a better robot skill. Loss sign and scale depend on the implementation.

Diagnostics: is the optimizer behaving?

Recorded update diagnostics — absent fields are explicit



Entropy, clipping, explained variance and timing are diagnostics — not a standalone pass/fail certificate.

Metric matrix: numerical audit of this trace · 1/3

- 01 approximate_kl: first 2.824 → final 0.03259; min 0.01741, max 2.824; 1024 logged samples.
- 02 clip_fraction: first 0.8713 → final 0.3572; min 0.2467, max 0.8713; 1024 logged samples.
- 03 entropy: first 1.830 → final -0.3148; min -0.3150, max 1.876; 1024 logged samples.
- 04 explained_variance: first 0.8329 → final 0.6748; min -1.234, max 0.9469; 1024 logged samples.

Metric matrix: numerical audit of this trace · 2/3

- 05 mean_loss: first 0.1724 → final -0.02380; min -0.03384, max 0.1724; 1024 logged samples.
- 06 mean_raw_return: first 8637 → final 9686; min 87.22, max 1.008e+4; 1738 logged samples.
- 07 mean_reward: first 0.03103 → final 0.03508; min 0.02922, max 0.03661; 1024 logged samples.
- 08 policy_loss: first 0.1711 → final -0.02410; min -0.03457, max 0.1711; 1024 logged samples.

Metric matrix: numerical audit of this trace · 3/3

09

value_loss: first 0.001326 → final 0.0003019; min 0.0002014, max 0.01952; 1024 logged samples.

Verification: define success before looking · 1/2

- 01 Require every episode to complete 600 samples without termination, with finite physical metrics and upright fraction ≥ 0.95 .
- 02 Require commanded fraction ≥ 0.50 , mean directed speed ≥ 0.35 m/s, signed intended displacement ≥ 2.0 m, and speed ratio ≥ 0.35 .
- 03 Require at least 8 alternating exclusive-support switches, each foot air fraction ≥ 0.08 , contact fraction ≥ 0.10 , and lift/touch events.
- 04 Require both-feet aerial fraction ≥ 0.03 ; this distinguishes running from fast grounded stepping.

Verification: define success before looking · 2/2

05

Use evaluator version 2 signed displacement, which anchors intended heading at command changes and advances it only by commanded yaw.

06

Require zero, random, and loaded initial controls to fail the final gate; verify hashes, finite telemetry, 61-to-14 ONNX, parity, and video.

Results: what was actually measured • 1/3

- 01 Actual final evidence: PASS for running-e2e-20260907-v4; Studio evaluation passed 16/16 complete 12-second episodes.
- 02 Independent seeds 501-505 passed 5/5; directed speeds were 0.7382723265470258-0.7746384824233077 m/s.
- 03 Signed intended displacement was 3.8483205827178626-9.01295603365732 m; speed ratios were 0.9843631020627012-1.0328513098977434.
- 04 Aerial fractions were 0.35-0.3983333333333333, with 167-170 support switches; every episode had upright fraction 1.0.

Results: what was actually measured · 2/3

- 05 The v3 initializer was already aerial at fraction 0.275 with 156 switches and 0.6664 m/s, but failed with -0.2392 m intended displacement.
- 06 After 524,288 continuation transitions, comparison seed 501 passed; the final checkpoint retained a pass with raw return 11467.54.
- 07 Final raw-return change versus initialization was 1683.1782192444243; learned tensor L2 change was 7.672545078668774.
- 08 ONNX/native maximum action error was 1.430511474609375e-6; zero and random controls fell after 43-49 steps and failed the gate.

Results: what was actually measured · 3/3

09

Final-100 means: policy_loss -0.022870965473121032, value_loss 0.0003489635748883302, mean_loss -0.022522001879488017.

10

Final-100 approximate_kl 0.03082194794646057, clip_fraction 0.33925048828125, explained_variance 0.5853159072389115.

Watch the actual saved rollout



BEFORE PLAY

Predict one success signal and one failure signal.

AFTER PLAY

Which claims does this clip support? What remains untested?

12.0 s saved MP4 • Click in PowerPoint Slide Show • Backup: videos/running-rollout.mp4

Limits and failure analysis · 1/2

- 01 The policy specializes to one nominal 0.75 m/s forward command; arbitrary speed, turning, and disturbance recovery are untested.
- 02 The passing paths still curve; the result proves signed intended progress under the gate, not straight-line path perfection.
- 03 The base acquired running, while v4 improved intended progress; crediting v4 with discovering the entire gait would be inaccurate.
- 04 Raw actor means can exceed the applied $[-4, 4]$ target-offset range, so nominal XML success is not a hardware command-quality certificate.

Limits and failure analysis · 2/2

05

No BAM dynamics, current limits, latency, randomized physics, terrain, pushes, battery, thermal, cable, backlash, or hardware safety tests.

06

One training lineage and five nominal audit seeds do not establish training-seed robustness or broad generalization.

Student lab: change one variable, keep the gates · 1/2

- 01 Recommended exercise, not saved evidence: classify a 0.72 m/s gait with aerial fraction 0.01 against the exact running gate.
- 02 Recommended exercise, not saved evidence: identify observable yaw-rate inputs and explain why absolute-heading reward is invalid.
- 03 Recommended exercise, not saved evidence: clip the recorded first raw actor vector and list saturated joints.
- 04 Recommended exercise, not saved evidence: construct a circular trajectory with high body-frame speed but near-zero intended displacement.

Student lab: change one variable, keep the gates · 2/2

05

Recommended exercise, not saved evidence: predeclare held-out speed and yaw-rate commands for interpolation and extrapolation tests.

Submit an experiment, not just a video

- 01** Before training: write one hypothesis, the changed parameter, the fixed baseline, seed list, acceptance criteria and time/transition budget.
- 02** During training: save the exact command, environment versions, initialization, checkpoint and raw reward/loss/KL metrics. Never overwrite the saved reference run.
- 03** After training: evaluate the frozen exported policy on matched seeds; include all failures, aggregate metrics, the full rollout video and a source-bound result table.
- 04** Rubric / 10: reproducibility 2; environment and reward explanation 2; PPO and metrics interpretation 2; honest verification 2; one evidence-based next experiment 2.

Source map and further study · 1/2

01

AGENTS.md

02

microduck_local/AGENTS.md

03

microduck_local/src/microduck_local/contract.py

04

README.md

Source map and further study · 2/2

05

`docs/microduck-training-student-book/chapters/02a-ppo-code-lab.md`

06

`docs/microduck-training-student-book/chapters/02b-gae-workshop.md`

07

`scripts/scenarios/README.md`

08

`README.md`